

CS 440: Introduction to Artificial Intelligence

Assignment 1: Fast Trajectory Planning

Mary Grlic
Soraya Mosavi

Due: February 22, 2026

Part 0

We implemented grid generation using a randomized DFS-style traversal. Each cell is assigned as blocked with probability 0.3 and unblocked otherwise. We generated 50 distinct 101×101 grid maps and saved them to files for consistent experimentation. The maps can be loaded and visualized for testing the search algorithms.

Part 1

Initially, the agent does not know which cells are blocked, so it assumes all cells are free (unblocked). Since the agent uses the Manhattan distance, it computes the shortest path to another cell assuming there are no obstacles. In Figure 8, moving east and north both have equal f -values, and A* must break the tie between the two nodes. To do so, the algorithm favors the cell with a larger g -value, as stated in the detailed project description.

An agent in a finite gridworld will either reach its target or discover that this is impossible in finite time. Since the grid is finite, there are a finite number of cells. During the search process, the agent updates its knowledge whenever it encounters a blocked cell. Each cell is discovered at most once. Because the number of cells is finite and each A* search expands only finitely many states, the agent will either reach the target or determine that it is unreachable in finite time.

Part 2

We compared Repeated Forward A* using two tie-breaking rules when f -values are equal: favoring smaller g versus favoring larger g . On our 50 generated 101×101 maps, both approaches solved 26 maps. However, for the 26 maps solved by both approaches, the large- g tie breaker expanded fewer nodes on every single map (26 wins, 0 ties).

On average, small- g expanded 267,980.5 nodes while large- g expanded 9,261.08 nodes. This suggests that when f -value ties occur, preferring larger g tends to reduce search effort in this repeated replanning setting. Likely, this is because it commits more strongly to deeper partial paths instead of exploring many shallow alternatives.

Part 3

For Part 3, we compared Repeated Forward A* and Repeated Backward A* on the same 50 generated 101×101 maps, using the large- g tie-breaking rule (since it performed better in Part 2). Both algorithms were able to solve 26 out of the 50 maps, and they solved the exact same 26 maps.

However, there was a significant difference in the number of expanded nodes. On average:

Forward A* expanded $\approx 9,261$ nodes

Backward A* expanded $\approx 129,930$ nodes

Forward A* expanded fewer nodes on all 26 maps that were solved by both algorithms, with no ties. This demonstrates that, in our experiments, planning from the agent's current position toward the goal (Forward A*) was substantially more efficient than planning backward from the goal to the agent. Although both methods found paths on the same maps, Forward A* required significantly less search effort.

Part 4

Manhattan distances are consistent in gridworlds in which the agent can move only in the four main compass directions. The Manhattan distance between two points (x_1, y_1) and (x_2, y_2) is defined as:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Since the agent can move only up, down, left, or right (no diagonal moves), each action changes the Manhattan distance by at most 1. Therefore, for every node n and successor n' :

$$h(n) \leq c(n, a, n') + h(n')$$

where $c(n, a, n') = 1$ for each valid move. Thus, the Manhattan distance satisfies the consistency condition.

A heuristic is consistent if and only if, for every node n and successor n' :

$$h(n) \leq c(n, a, n') + h(n')$$

and

$$h(\text{goal}) = 0.$$

In Adaptive A*, heuristic values are updated according to:

$$h_{\text{new}}(n) = g^*(\text{goal}) - g(n),$$

where $g^*(\text{goal})$ is the optimal path cost to the goal and $g(n)$ is the cost from the start to node n .

Because shortest-path distances satisfy the triangle inequality:

$$g^*(\text{goal}) \leq g(n) + c(n, n') + h_{\text{new}}(n'),$$

it follows that:

$$h_{\text{new}}(n) \leq c(n, n') + h_{\text{new}}(n').$$

Even if action costs increase, the updated heuristic values remain bounded by the true shortest-path distances. Therefore, Adaptive A* preserves consistency.

Part 5

For Part 5, we ran Adaptive A* and Repeated Forward A* (using the large- g tie breaker) on the same 50 generated 101×101 maps. Both algorithms solved 26 maps, and they solved the exact same 26 maps.

On average:

Forward A* expanded 9,261.08 nodes

Adaptive A* expanded 9,155.62 nodes

Adaptive A* expanded fewer nodes on 25 of the 26 maps that both algorithms solved (Forward only won once, with no ties). This is expected because Adaptive A* reuses information from previous searches by updating heuristic values, leading to fewer expansions in later replanning phases.

Part 6

To determine whether the performance difference between Repeated Forward A* and Adaptive A* is statistically significant, we formulate the following hypotheses:

H_0 : There is no difference in the mean number of expanded nodes.

H_1 : Adaptive A* expands significantly fewer nodes.

Since both algorithms were evaluated on the same 50 maps, the data can be treated as paired samples. For each map, we compute the difference:

$$d_i = (\text{Forward}_i - \text{Adaptive}_i).$$

We then compute the mean and standard deviation of the differences and apply a paired t -test:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}.$$

If the resulting p -value is smaller than a chosen significance level (e.g., $\alpha = 0.05$), we reject the null hypothesis and conclude that the performance difference is statistically significant. Otherwise, we conclude that the observed difference may be due to sampling variability.